European Network of
Transmission System Operators
For Electricity

Public Interface

Energy Communication Platform
Version 4.16.0

Publication Date: 06.05.2025

Energy Communication Platform, version 4.16.0

This software is protected by copyright.

Copyright © 2025, ENTSO-E AISBL
All rights reserved.

This software was developed for ENTSO-E by Unicorn Systems a.s.
www.unicorn.com

Licenses and third-party components:

- This product includes software developed by the Apache Software Foundation (https://www.apache.org/)
- This product includes following third-party open-source components:
  - Apache Tomcat (https://tomcat.apache.org/)
  - Spring Boot (https://spring.io/projects/spring-boot/)
  - ActiveMQ Artemis (https://activemq.apache.org/components/artemis/)
  - Qpid JMS (https://qpid.apache.org/components/jms/)
  - Apache Camel (https://camel.apache.org/)
  - Logback (https://logback.qos.ch/)
  - Spring Framework (https://spring.io/projects/spring-framework/)
  - Hibernate (https://www.hibernate.org/), licensed under LGPL 2.1
  - AngularJS (https://angularjs.org/)
  - Liquibase (https://www.liquibase.org/)

1. Contents

1. Contents ... 3
2. Abstract ... 6
3. Revision History ... 7
4. Overview ... 9
4.1. Messaging API ... 9
4.1.1. Send Message ... 9
4.1.2. Receive Message ... 9
4.1.3. Confirm Receive Message ... 10
4.1.4. Check Message Status ... 10
4.1.5. Connectivity Test ... 10
4.2. Monitoring API ... 10
4.3. Management API ... 11
4.3.1. Message Paths Management ... 11
4.4. Plugins ... 11
4.4.1. Receive Handlers ... 11
4.4.2. Send Handlers ... 12
4.4.3. Archive Handlers ... 12
5. Messaging API - Logical Description ... 14
5.1. Send Message ... 14
5.1.1. Input/Output Description ... 15
5.2. Receive Message ... 16
5.2.1. Input/Output Description ... 17
5.3. Confirm Receive Message ... 19
5.3.1. Input/Output Description ... 19
5.4. Check Message Status ... 20
5.4.1. Input/Output Description ... 21
5.5. Connectivity Test ... 23
5.5.1. Input/Output Description ... 24
5.5.2. Usage of the Connectivity Test Operation ... 25
6. Messaging API - Web Services ... 26

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

6.1. Implementation of the Logical API 26
6.2. Webmethods description 26
6.2.1. Webmethod SendMessage 26
6.2.2. Webmethod ReceiveMessage 27
6.2.3. Webmethod ConfirmReceiveMessage 29
6.2.4. Webmethod CheckMessageStatus 30
6.2.5. Webmethod ConnectivityTest 31
6.3. Error Handling 32
7. Messaging API - File System Shared Folder (FSSF) 33
7.1. File Naming Conventions 33
7.2. Folders 35
7.3. Message Send-Receive Scenario 36
8. Messaging API - AMQP 37
8.1. Send Message 37
8.1.1. Input Message Structure 38
8.1.2. Output Message Structure 38
8.1.3. Fault Message Structure 39
8.2. Receive Message 40
8.2.1. Message Structure 40
8.3. Check Message Status 41
8.3.1. Message Structure 41
9. Management API Description 43
9.1. Message Paths Management 43
9.1.1. Create Message Path 43
9.1.2. Get Message Path List 45
9.1.3. Edit Message Path 46
9.1.4. Delete Message Path 47
9.2. Error Handling 48
10. Plugins 49
10.1. Receive Event Handler 49
10.2. Send Event Handler 51
10.3. Archive Event Handler 52
10.4. Java API for Plugins 55

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

10.4.1. Receive Message Method 55
10.4.2. Check Message Status Method 56
10.4.3. Error Handling 57
11. Configuration of Channels and Message Receivers 59
12. Attachment A - Web Service Public API WSDL 60
13. Figure Content 61

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

2. Abstract

This document describes the public API of the ECP endpoint through which business applications can integrate with ECP and use it for sending and receiving messages.

The document covers all basic ECP messaging functions and describes in detail the nuances of all three technological channels – Web Service API, AMQP API and file system shared folder (FSSF). In addition, more advanced functionalities are presented, including the ECP health monitoring API and the ECP extension API, including the send handlers and the receive handlers.

This document is primarily targeted at Administrators and Developers.

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

3. Revision History

Version	Date	Author	Description
1.00	02/07/2009	Stanislav Mikulecký	Final Revision
1.5	16/02/2009	Jiří Neuman	Changes for ECP 1.5
2.1	03/02/2011	Jiří Dudek	Changes for ECP 2.1
3.0	04/07/2013	Luboš Světik, Petr Zdráhal	Changes for ECP 3.0
3.0.1	02/08/2013	Petr Zdráhal	RTE remarks
3.0.2	27/08/2013	Unicorn	English revision
4.0.0	03/17/2016	Jiří Dudek, Jiří Neuman	Changes for ECP 4.0
4.0.1	06/07/2016	Jiří Dudek	Minor updates
4.0.2	22/07/2016	Jiří Neuman	Added management API
4.1.3	15/11/2017	Laura Betcherová	Aligned to ECP 4.1.3
4.1.4	05/02/2018	Laura Betcherová	Documentation review
4.2.0	21/03/2018	Josef Brož	Version update
4.3.0	26.06.2018	Josef Brož	Version update
4.3.0	05.10.2018	Barbora Klabačková	File Naming Conventions update
4.4.0	14.06.2019	Josef Brož	Version update
4.5.0	22.08.2019	Jiří Kakrda	Version update
4.6.0	05.11.2019	Jiří Kakrda	Version update
4.6.1	18.03.2020	Anna Langrová	Version update
4.7.0	28.06.2020	Lukáš Voříšek	Version update
4.7.1	31.07.2020	Anna Langrová	FSSF write/read handling

4.8.0	14.05.2021	Lukáš Voříšek	Webservice receive method input corrected
4.8.1	09.07.2021	Lukáš Voříšek	Version update
4.8.2	03.12.2021	Daniel Janota, Jiří Kakrda	FSSF default extension, formatting
4.9.0	26.05.2022	Jiří Kakrda	Check Message Status data
4.10.0	07.11.2022	Daniel Janota	Hibernate
4.11.0	16.05.2023	Jiří Kakrda	Version update
4.12.0	27.10.2023	Jiří Kakrda	Version update
4.14.0	10.06.2024	Jiří Kakrda	Plugins chapter update
4.15.0	21.10.2024	Jiří Kakrda	AMQP send message required class
4.16.0	06.05.2025	Jiří Kakrda	Version update

4. Overview

The public interface of ECP can be divided into two groups according to the purpose of provided methods.

Messaging API – provides the key functions of ECP, which are sending and receiving messages, checking message delivery status and connectivity tests. ECP provides these functions via several channels.

Management API – provides management functions to administer the endpoint's configuration.

Monitoring API – provides support functions which report information about the current internal state of the deployed ECP module.

Plugins – the behaviour of ECP can be expanded by specific plugins that can modify how messages are sent and received.

4.1. Messaging API

Messaging API contains the core functions of ECP. They ensure the secure and reliable transportation of messages from one ECP endpoint to another.

4.1.1. Send Message

This function accepts a business message to send via ECP to another endpoint. The message is first validated and, if successful, accepted for transportation over ECP. After acceptance, ECP is responsible for the delivery of the message. As a result, a unique message ID is returned. This message ID can be used for further checks on the message delivery status.

4.1.2. Receive Message

This method receives a message which was delivered to the ECP endpoint. The method returns the message to the caller business application. The message is not marked as delivered until the application confirms the reception using the "Confirm Receive Message" method.

4.1.3. Confirm Receive Message

This method confirms to ECP that it correctly received a message. The method marks the message as received within ECP. At this point, ECP considers the message successfully transported.

4.1.4. Check Message Status

This method checks the status of a message (specified by a message ID), allowing the sender of the message to check whether the message reached its destination, whether there was an error during transportation or whether the message is waiting in some intermediate ECP component between the sender and the receiver endpoints.

4.1.5. Connectivity Test

To check the connectivity with another ECP endpoint, a tracing message can be sent using the connectivity test method. This method sends a tracing message and returns the message ID assigned by ECP. To verify that the tracing message reached the receiver, the sender should check its status.

Note that the connectivity test is meant to be used only for checking the connectivity from one ECP endpoint to another. You can think of the connectivity test as a "trace route" utility. You send a tracing message and its path is available using the Check message status method.

To check the connectivity between a business application and an ECP endpoint, refer to the Monitoring API description below.

4.2. Monitoring API

The monitoring interface can be used to check the health of a deployed ECP module. A business application can use the Monitoring API to verify that the ECP Endpoint to which it is connected is running and in a good shape.

The ECP administrator can discover various errors, including incorrectly deployed components or problems with the delivery of messages (stuck messages).

Monitoring functions are currently available through a JMX interface and Jolokia, which is an HTTP bridge for the JMX interface (see https://jolokia.org/).

4.3. Management API

ECP provides a SOAP web service interface for management of its configuration. The areas that can be controlled via the management interface are described below.

4.3.1. Message Paths Management

ECP endpoint gives the administrator a full set of operations over the message paths via the management API. Functions available to the administrator via message paths management are:

- fetch a list of all message paths defined for the endpoint
- create a new message path
- edit an existing message path
- remove an existing message path

4.4. Plugins

An ECP plugin is a class implementing a specific interface packed in JAR and put into a specific directory located in the deployed ECP module. ECP currently supports three types of plugins described in the following paragraphs.

The configuration of plugins is described in the *ECP Administration Guide*.

4.4.1. Receive Handlers

A receive handler is a plugin which is notified by ECP whenever a message of a given type is delivered to the endpoint in which the plugin is registered. When the receive handler is notified about a new message, it decides what to do with the message. In a typical implementation, the handler receives the message and passes it to a business application via Web service.

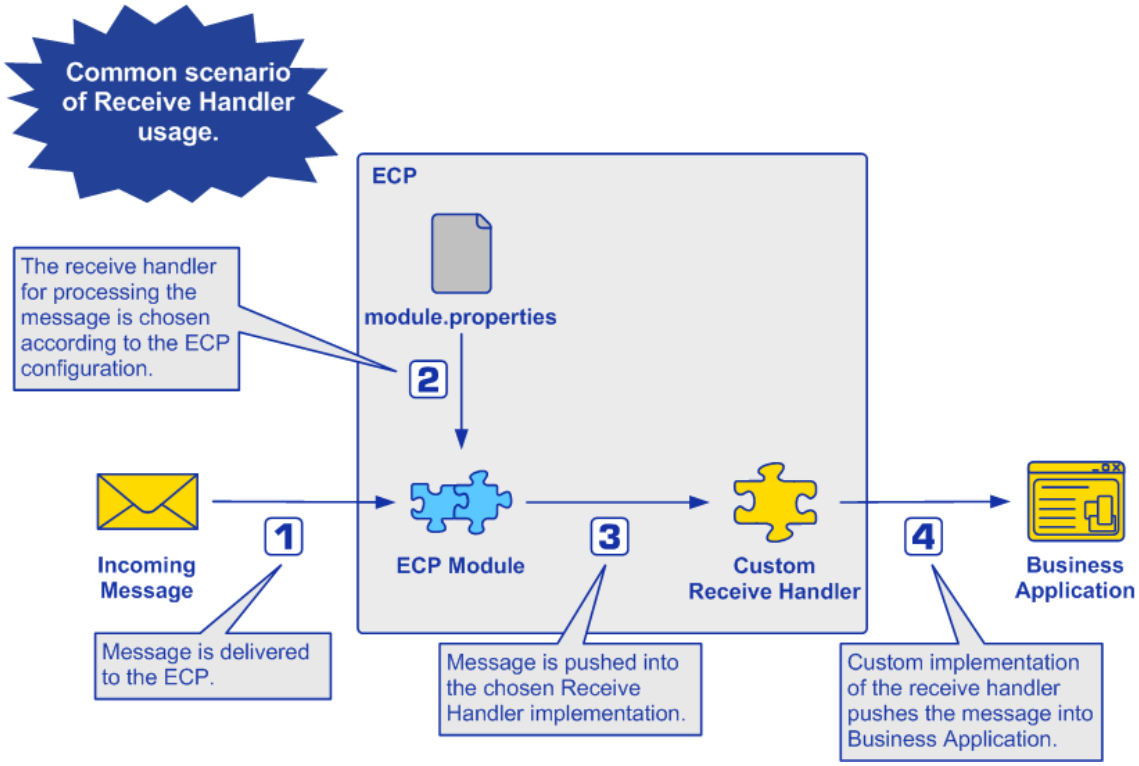

Figure 1: Receive Handlers

4.4.2. Send Handlers

A send handler is used for processing messages immediately after their status is changed to DELIVERED, RECEIVED or FAILED.

SendEventHandler is a custom Java class which is registered at the ECP endpoint. This class exposes one method called by ECP endpoint immediately after the status of a message of a given type changes. Note that ECP only notifies the handler about that message and it is up to the handler implementation as to how to process this notification. Typically, the send handler notifies a business application about the state change of a message it has sent, so that the application knows whether the message was or was not correctly received by the recipient.

4.4.3. Archive Handlers

An archive handler is a plugin responsible for passing messages and their metadata to archive. The archive handler implements the bridge between ECP and the targeted archive location (e.g. file system, database or a remote system connected through an AMQP queue).

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

Archive handler is a custom Java class implementing *ArchiveEventHandler* interface, which plugs into ECP endpoint. The handler gets called by ECP endpoint when there is a message content or metadata ready for moving to archive. It is up to the implementation of archive handler to pass the message content or metadata to the message store in required structure. Refer to chapter *Archive Event Handler* to see more details about the interface of archive handler.

5. Messaging API - Logical Description

This chapter describes the parameters and results of the messaging API methods. Its aims to provide an overview of the messaging API, which is the same for all channels. The principle is the following:

1. A message is sent by a business application using the Send Message operation.
2. The message is transported through the ECP network to the receiver.
3. The message is received by a business application using the Receive Message operation.
4. Message reception is confirmed by the receiving business application using the Confirm Message Receive operation and the message is marked as received.
5. The status of the message can be observed by the sending business application with the Check Message Status operation at any time.

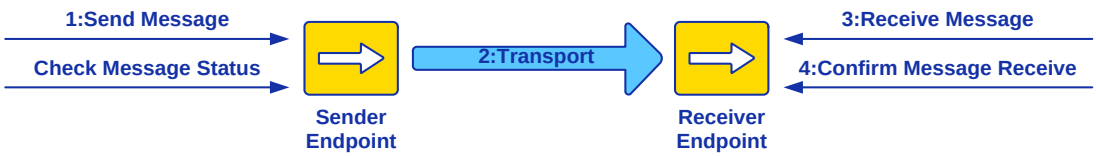

Figure 2: Messaging API - Logical Description

5.1. Send Message

This operation accepts and sends a business message to another endpoint. The message is validated and, if the validation passes, the message is accepted for transportation over ECP. After the message has been accepted, ECP is responsible for the delivery of the message. As a result of this method call, a unique message ID is returned. This message ID identifies the sent message and can be used to check the state of the message delivery at any further time.

This operation prevents double sending. The application should provide a conversation ID which must be unique for each message. In the case a conversation ID is reused and that the message was already sent, ECP does not send it again and returns the message ID back to the application.

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

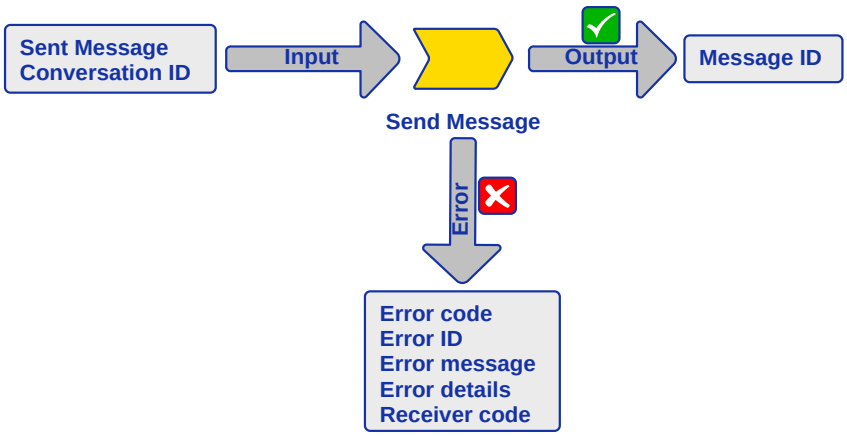

Figure 3: Send Message

5.1.1. Input/Output Description

INPUT PARAMETERS:

Parameter	Type	Description	Required
Sent Message	SentMessage	Metadata and payload of message.	True
Conversation ID	String	The ID which identifies this conversation. If send message is called twice with the same Conversation ID, it returns the same result. It is used to prevent double messages in the case of sender application error.	False

SENTMESSAGE OBJECT:

Parameter	Type	Description	Required
Receiver Code	String	Recipient's endpoint code. Regexp pattern: *[A-Za-z0-9-@]+*	True
Business Type	String	Message business type. Regexp pattern: *[A-Za-z0-9-]+*	True
BA Message ID	string	Message ID assigned by BA (optional). Regexp pattern: *[A-Za-z0-9-]**	False

Sender Application	string	Code or description of sender's BA (optional). Regexp pattern: *[A-Za-z0-9-]**	False
Content	content	Message payload (required). The type is different in all channels.	True

RESPONSE:

Parameter	Type	Description	Required
Message ID	string	Unique message ID that ECP automatically assigned to send message.	True

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the type of error (e.g. validation error, unexpected error, etc.).	True
Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True
Error message	string	An error message usually describing the reason of failure.	True
Error details	string	Additional error details.	False
Receiver code	string	Code of the receiver to which the message was addressed.	False

5.2. Receive Message

This operation receives a message delivered to the ECP endpoint. The method returns the message (if any) to the business application. The ECP message status is not changed to "delivered" until the "Confirm receive" method is called by the business application. This prevents the loss of messages in the case of a business application or network error.

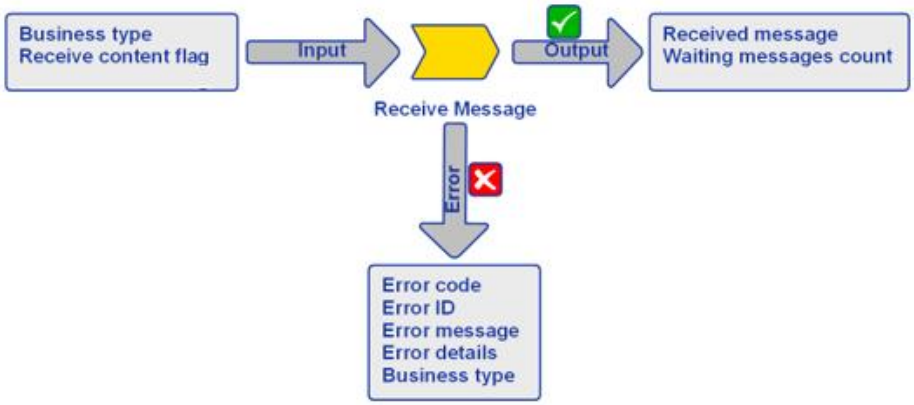

Figure 4: Receive Message

5.2.1. Input/Output Description

INPUT PARAMETERS:

Parameter	Type	Description	Required
Business type	String	The type of message to be received. When message type is not set, the first message waiting for download will be received, regardless of its type. Regexp pattern: *[A-Za-z0-9-]**	False
Receive content	boolean	The flag indicating whether the content of the message should also be downloaded. true – complete message is returned false – only metadata of message are returned (without content)	True

RECEIVEDMESSAGE OBJECT:

Parameter	Type	Description	Required
Message ID	string	Unique message ID.	True
Receiver Code	string	Endpoint code of the message's receiver.	True

Sender Code	string	Endpoint code of message's sender.	True
Business Type	string	Message business type.	True
BA Message ID	string	Message ID assigned by BA.	False
Sender Application	string	Core or description of sender's BA.	False
Content	content	Message payload (required). It can be null if the "Receive content" flag is set to false. The type is different in all channels.	False

RESPONSE:

Parameter	Type	Description	Required
Received message	ReceivedMessage	Metadata and payload of message. Can be null if there are no messages to be received.	False
Waiting messages count	number	Number of remaining messages of the business type that was specified as the input parameter of ReceiveMessage method.	True

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the type of error (e.g. validation error, unexpected error, etc.).	True
Error ID	string	Unique identification of the failure. The Error ID shall be always used to track the error in the application logs.	True
Error message	string	Error message usually describing the failure reason.	True
Error details	string	Additional error details.	False
Business type	string	Business type used as a parameter.	False

5.3. Confirm Receive Message

A business application uses this operation to confirm to ECP that a message has been correctly received. The ECP message state is then changed to "received". At this very moment, ECP considers the message successfully transported.

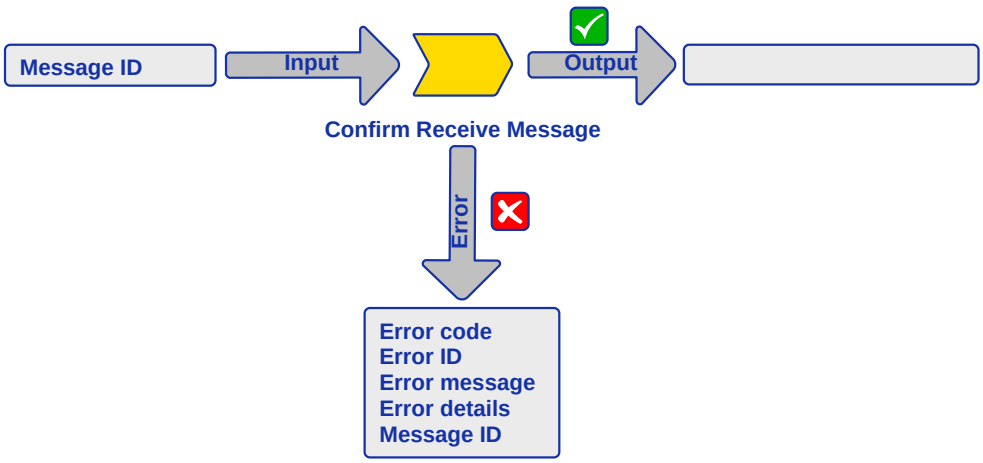

Figure 5: Confirm Receive Message

5.3.1. Input/Output Description

INPUT PARAMETERS:

Parameter	Type	Description	Required
Message ID	string	Unique message ID. Regexp pattern: *[A-Za-z0-9-]+*	True

RESPONSE:

No response.

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the type of error (e.g. validation error, unexpected error, etc.).	True

Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True
Error message	string	Error message usually describing the failure reason.	True
Error details	string	Additional error details.	False
Message ID	string	ID of message which should be confirmed.	False

5.4. Check Message Status

This operation returns the state of the message (specified by message ID). This method allows the sender of the message to check whether the message reached its destination, whether there was an error during transportation or if the message is waiting in some intermediate ECP component.

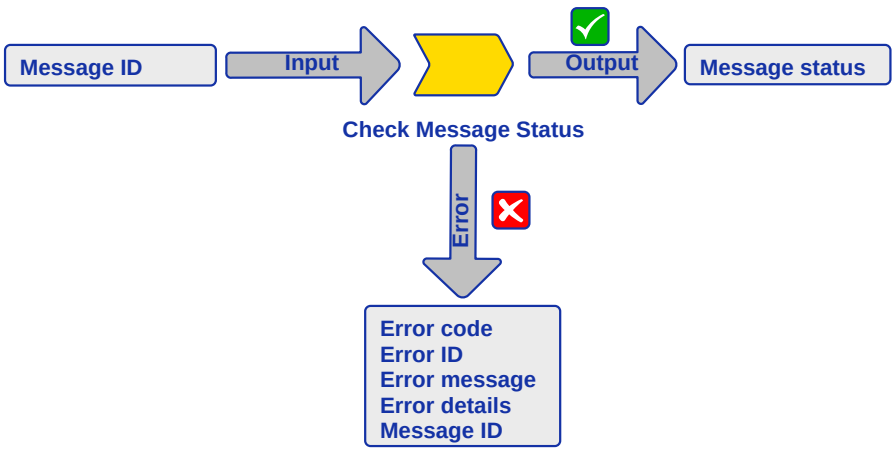

Figure 6: Check Message Status

5.4.1. Input/Output Description

INPUT PARAMETERS:

Parameter	Type	Description	Required
Message ID	string	Unique message ID. Regexp pattern: *[A-Za-z0-9-]+*	True

RESPONSE:

Parameter	Type	Description	Required
Message status	MessageStatus	Metadata processing status.	False

MESSAGESTATUS OBJECT:

Parameter	Type	Description	Required
Message ID	string	Unique message ID.	True
State	MessageState	State of the message.	True
Receiver Code	string	Endpoint code of the message's receiver.	True
Sender Code	string	Endpoint code of the message's sender.	True
Business Type	string	Message business type.	True
BA Message ID	string	Message ID assigned by BA (optional).	False
Sender Application	string	Code or description of a sender's BA (optional).	False
Send timestamp	timestamp	Exact date and time the message was sent.	True
Receive timestamp	timestamp	Exact date and time the message was received.	False
Trace	List of MessageTraceItem.	Detailed information about the message's course through ECP network.	False

MESSAGETRACEITEM OBJECT:

Parameter	Type	Description	Required
Timestamp	timestamp	Exact date and time of the event.	True
State	MessageTraceState	State of message processing.	True
Component	string	The code of ECP messaging component on which the event occurred.	True
Component description	string	Human-understandable description of the messaging component.	True
Details	string	Details about the event.	False

MESSAGESTATE ENUMERATION:

Name	Description
ACCEPTED	The message has been accepted by the ECP component and stored in the messagebox.
DELIVERED	The message has been delivered to the target endpoint.
RECEIVED	The message has been received by the target business application or user.
FAILED	The processing of the message failed.

MESSAGETRACESTATE ENUMERATION:

Name	Description
ACCEPTED	The message has been accepted by the ECP component and stored in the messagebox.
DELIVERED	The message has been delivered to the target endpoint.
RECEIVED	The message has been received by the target business application or user.
FAILED	The processing of the message failed.

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the type of the error (e.g. validation error, unexpected error, etc.).	True
Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True
Error message	string	Error message usually describing the failure reason.	True
Error details	string	Additional error details.	False
Message ID	string	ID of message whose state should be returned.	False

5.5. Connectivity Test

To check connectivity with another ECP endpoint, a tracing message can be sent using the connectivity test method. This method sends a tracing message and returns the message ID assigned by ECP. The tracing message is delivered to the receiver endpoint but is never passed to a receiver's business application. Business type is important parameter because ECP can deliver messages with different business types using different path through the ECP network. To verify that the tracing message reached the receiver endpoint, the sender can check the delivery status of the message using the "Check Message Status" method.

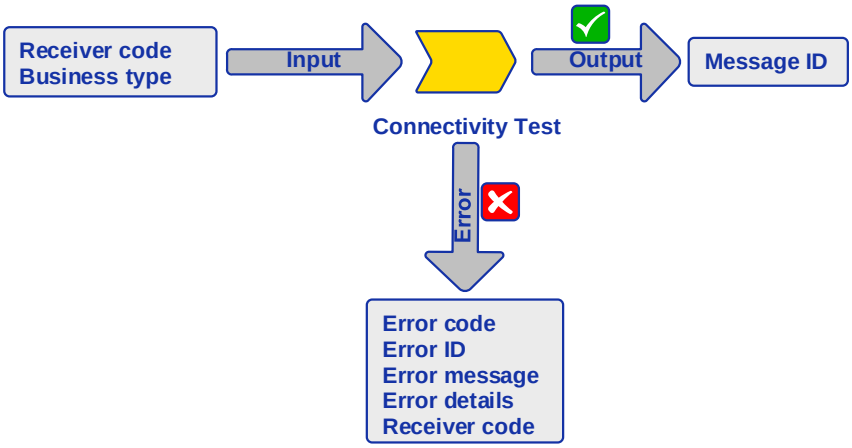

Figure 7: Connectivity Test

5.5.1. Input/Output Description

INPUT PARAMETERS:

Parameter	Type	Description	Required
Receiver code	string	Endpoint code of the receiver. Regexp pattern: *[A-Za-z0-9-@]+*	False
Business type	string	Message business type. Regexp pattern: *[A-Za-z0-9-]+*	True

RESPONSE:

Parameter	Type	Description	Required
Message ID	string	The unique message ID that ECP automatically assigned to a sent message.	True

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the error type (e.g. validation error, unexpected error, etc.).	True

Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True
Error message	string	Error message usually describing the reason of failure.	True
Error details	string	Additional error details.	False
Receiver code	string	Code of receiver to which was message addressed.	False

5.5.2. Usage of the Connectivity Test Operation

1. Call the Connectivity Test method, which sends a tracing message to the endpoint specified by the Receiver code parameter.
2. The response of the method contains the ECP message ID assigned to the tracing message. Remember this message ID.
3. To verify connectivity with the receiver, call the Check Message Status method with the message ID of the tracing message as a parameter.
4. If the message state is DELIVERED, the tracing message reached the receiver and the connectivity test is successful.
5. If the message is not DELIVERED after the first call of the Check Message Status operation, call it repeatedly. Also, check the message trace to find out which ECP components the message has passed through.
6. If the message state contains the value FAILED or if the message state does not change to DELIVERED after a while and the message trace does not contain new values, the connection may be broken: the connectivity test is not successful.

6. Messaging API - Web Services

Messaging functions are provided by the **ECPEnpointService** webservice (for WSDL, see Attachment A). When the ECP application is running, the WSDL is available at the URL *http(s):///<server IP adress>:<port>/ECP_MODULE/services/ECPEndpointService.wsdl*.

Depending on the configuration of the ECP endpoint, the webservice is available via HTTP or HTTPS protocol.

6.1. Implementation of the Logical API

The web service channel provides access to all methods of messaging API. This chapter describes the key concepts of the implementation:

- Input parameters are wrapped into the request objects.
- Results are wrapped into the response objects.
- Error description is transferred as Detail part of SOAP fault.

These types are described in the following web-method description.

6.2. Webmethods description

6.2.1. Webmethod SendMessage

This webmethod provides the Send Message operation described in chapter *Messaging API - Logical Description* > *Send Message*. A detailed description of webmethod types and their attributes may be found in Attachment A – Webservice Public API WSDL.

THE INPUT/OUTPUT WRAPPING TYPES:

Logical type	Wrapping type
Input	SendMessageRequest
Response	SendMessageResponse

Fault	SendMessageFault
Fault Detail	SendMessageError

Domain model of webmethod:

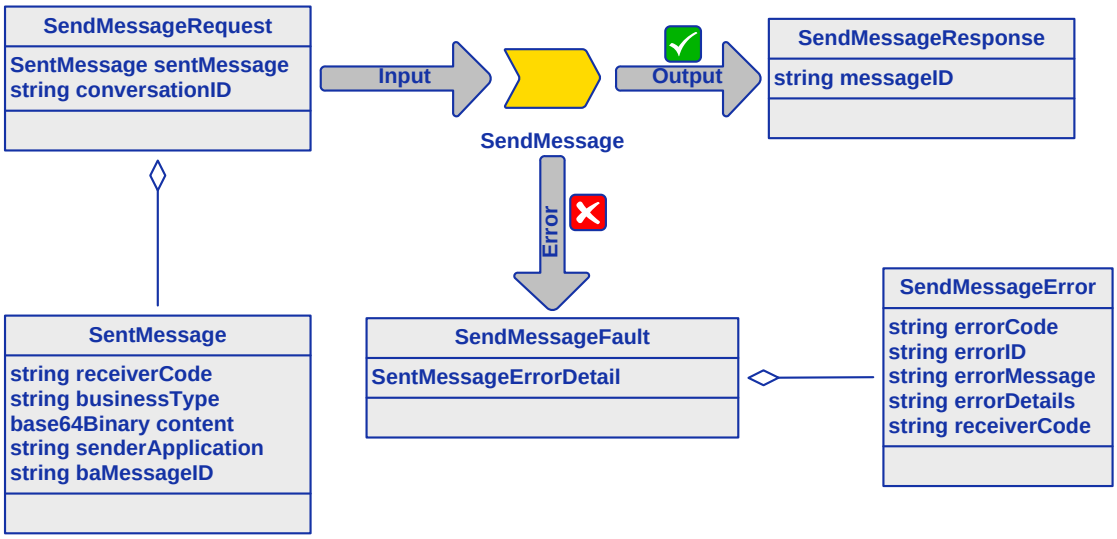

Figure 8: WebMethod Domain Model

6.2.2. Webmethod ReceiveMessage

This webmethod provides the Receive Message operation described in chapter *Messaging API - Logical Description > Receive Message*.

Note: Content received through Web Service is received as an attachment and not directly in the content field.

ERROR:

Parameter	Type	Description	Required
Error code	string	The code representing the type of error (e.g., validation error, unexpected error, etc.).	True
Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True

Error message	string	An error message usually describing the reason of failure.	True
Error details	string	Additional error details.	False
Receiver code	string	Code of the receiver to which the message was addressed.	False

Receive Message. A detailed description of the webmethod types and their attributes may be found in Attachment A – Webservice Public API WSDL.

THE INPUT/OUTPUT WRAPPING TYPES:

Logical type	Wrapping type
Input	ReceiveMessageRequest
Response	ReceiveMessageResponse
Fault	ReceiveMessageFault
Fault Detail	ReceiveMessageError

Domain model of webmethod:

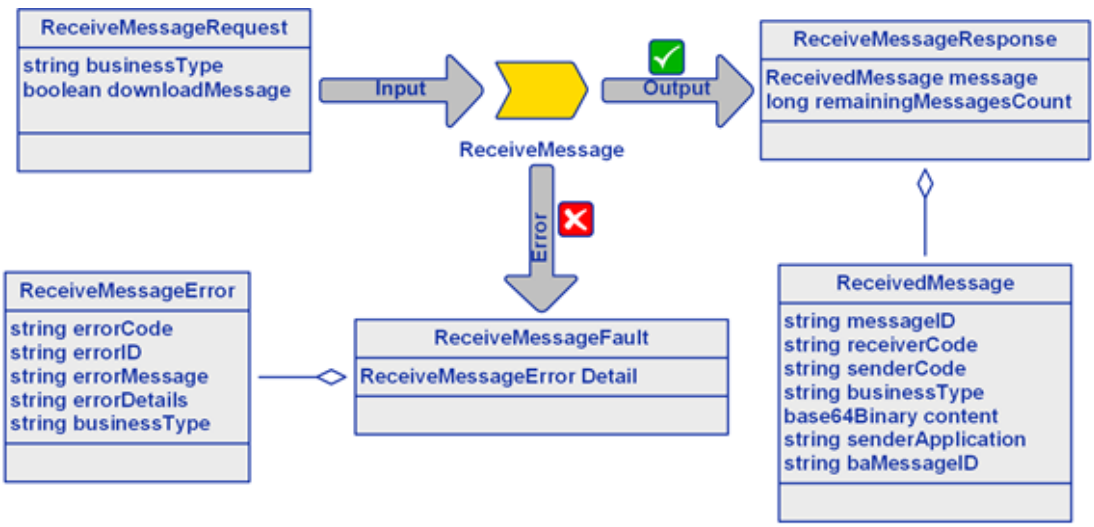

Figure 9: WebMethod Domain Model

6.2.3. Webmethod ConfirmReceiveMessage

This webmethod provides the Confirm Receive Message operation described in chapter *Messaging API - Logical Description > Confirm Receive Message*. A detailed description of the webmethod types and their attributes may be found in Attachment A – Webservice Public API WSDL.

THE INPUT/OUTPUT WRAPPING TYPES:

Logical type	Wrapping type
Input	ConfirmReceiveMessageRequest
Response	ConfrimReceiveMessageResponse
Fault	ConfirmReceiveMessageFault
Fault Detail	ConfirmReceiveMessageError

Domain model of webmethod:

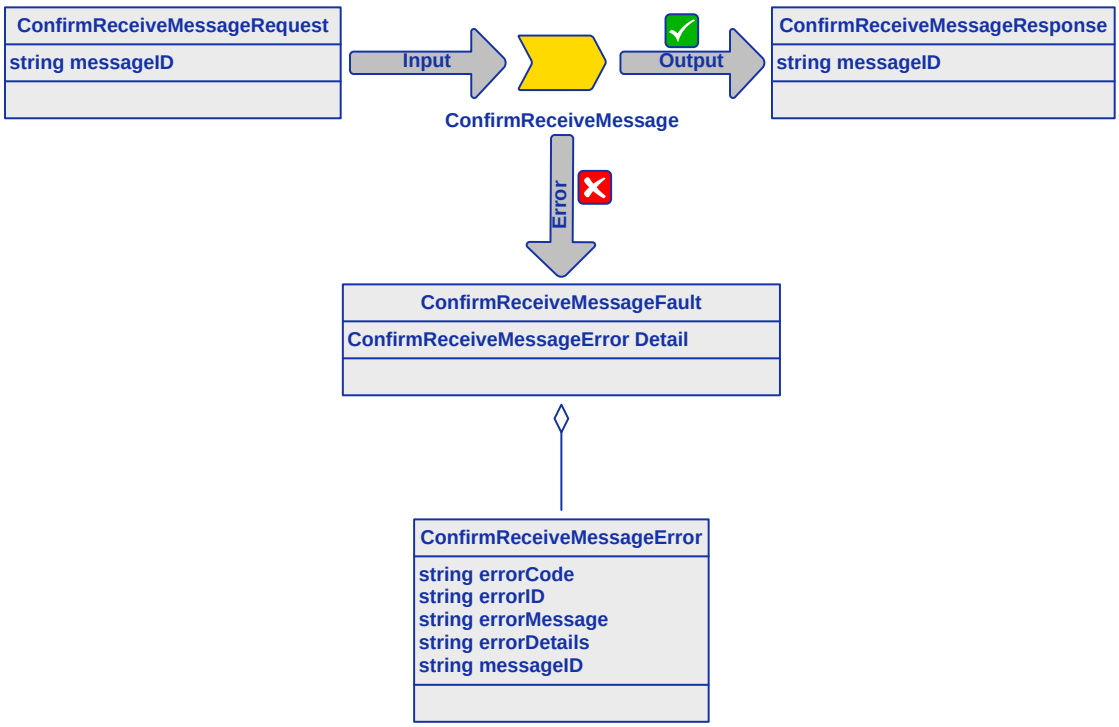

Figure 10: WebMethod Domain Model

6.2.4. Webmethod CheckMessageStatus

This webmethod provides the Check Message Status operation described in chapter *Messaging API - Logical Description* > *Check Message Status*. A detailed description of the webmethod types and their attributes may be found in Attachment A – Webservice Public API WSDL.

THE INPUT/OUTPUT WRAPPING TYPES:

Logical type	Wrapping type
Input	CheckMessageStatusRequest
Response	CheckMessageStatusResponse
Fault	CheckMessageStatusFault
Fault Detail	CheckMessageStatusError

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

Domain model of webmethod:

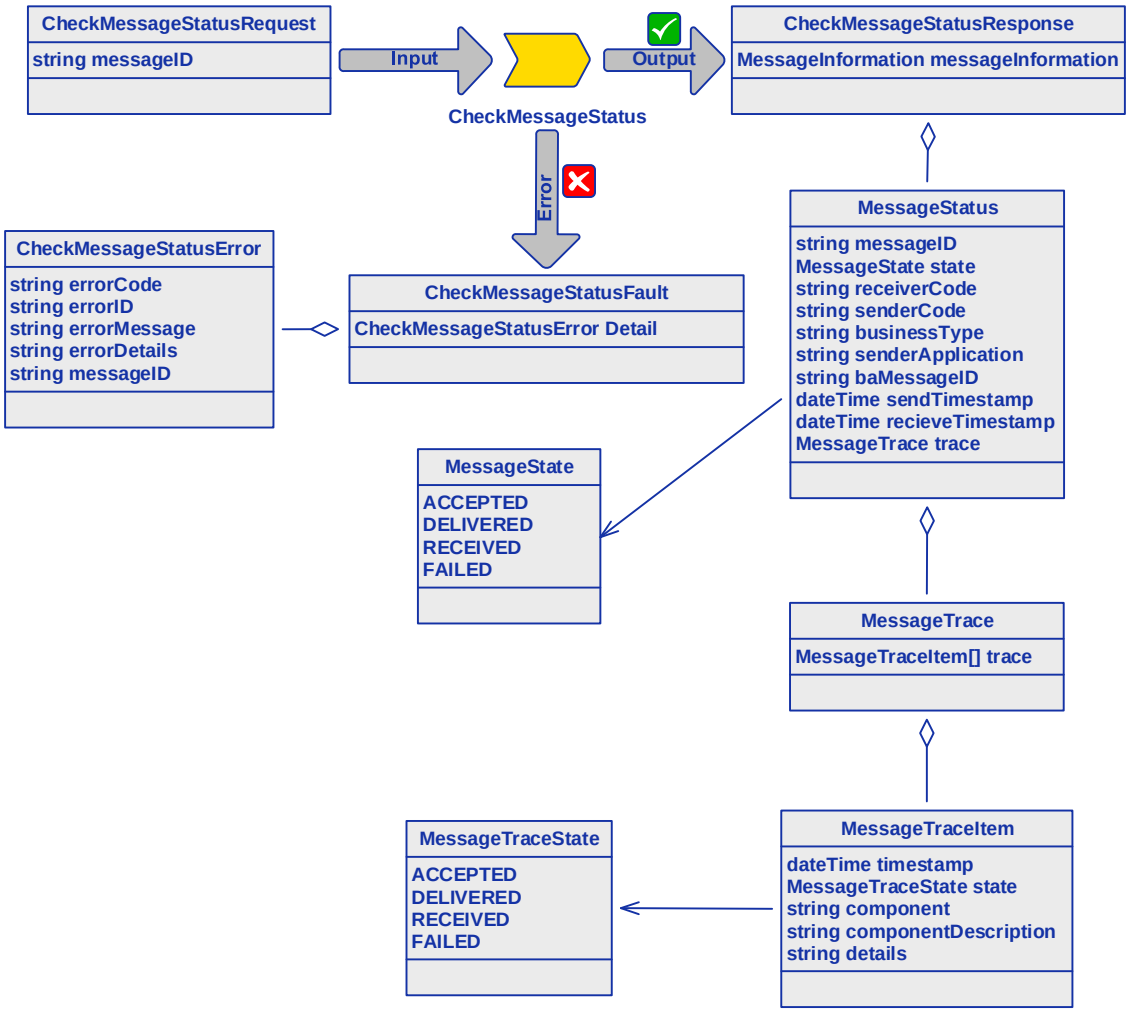

Figure 11: WebMethod Domain Model

6.2.5. Webmethod ConnectivityTest

This webmethod provides the Connectivity Test operation described in chapter *Messaging API - Logical Description > Connectivity Test*. A detailed description of the webmethod types and their attributes may be found in Attachment A – Webservice Public API WSDL.

The input/output wrapping types:

THE INPUT/OUTPUT WRAPPING TYPES:

Logical type	Wrapping type
Input	ConnectivityTestRequest
Response	ConnectivityTestResponse
Fault	ConnectivityTestFault
Fault Detail	ConnectivityTestError

Domain model of webmethod:

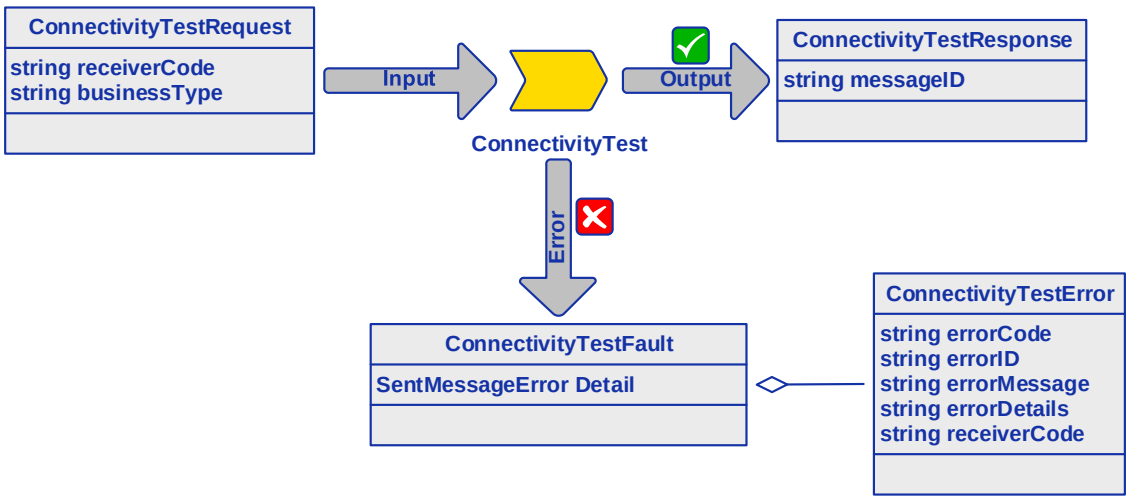

Figure 12: WebMethod Domain Model

6.3. Error Handling

Errors which occur on the server-side of web service (e.g. input data validation errors, unexpected errors, etc.) are reported back to the client using the standard SOAP Fault mechanism (http://www.w3.org/TR/soap12-part1/#soapfault). The ECP specific information about the error is transported using the Detail part of the SOAP Fault element. The fault and detail types are described in each webmethod description.

7. Messaging API - File System Shared Folder (FSSF)

The FSSF is an interface between ECP and the business applications that use files. The folders where the exchange occurs should be accessed by ECP as local files.

This interface allows a BA to send messages via an OUT folder, and received messages are filed via an IN folder. In addition, the ECP endpoint can be configured to create one-per-sent-message log files (stored into the OUT_LOG folder). If a message cannot be sent (due to a validation problem or an internal error), ECP moves it from the OUT folder to the OUT_ERROR folder.

Note that the FSSF channel only provides a subset of the messaging API operations and possibilities. The following operations are supported:

- Send message – without the support of the conversation ID
- Receive message – the "confirm message receive" operation is implicit. The message stated is changed to "received" immediately after the message is written in the IN folder.

Refer to the *ECP Administration Guide* for detailed information about FSSF configuration.

7.1. File Naming Conventions

There are four types of files used in the FSSF channel. Each file type is written into a separate folder and the file name is made of several parts separated by an underscore (underscores must exist even if optional parts are empty):

- file to send (OUT):
  <SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.EXT
- failed file (OUT_ERROR):
  <SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.EXT
- received file (IN):
  <SENDERAPP>_<SENDER>_<TYPE>_<BAMessageID>_<ECPMessageID>.EXT
- log file (OUT_LOG):
  <SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.log

File name part	Type	Description
<SENDERAPP>	Optional	Identifier of application of sender, for informational purposes only (supplied by sender BA). Regexp pattern: *[A-Za-z0-9-]**

<RECEIVER>	Mandatory	Code of endpoint of recipient (supplied by sender BA). Regexp pattern: *[A-Za-z0-9-@]+*
<SENDER>	Mandatory	Code of endpoint of sender. Regexp pattern: *[A-Za-z0-9-@]+*
<TYPE>	Mandatory	Type of message (supplied by sender BA). Regexp pattern: *[A-Za-z0-9-]+*
<BAMessageID>	Optional	The unique message identifier assigned by BA. It is recommended that GUID be used to avoid uniqueness conflicts in the OUT folder. Regexp pattern: *[A-Za-z0-9-]**
<ECPMessageID>	Mandatory	The unique GUID identifier assigned by ECP to identify the message. Its uniqueness ensures avoiding uniqueness conflicts in the IN folder. Regexp pattern: *[A-Za-z0-9-]+*
<EXT>	Optional	File extension (supplied by BA). ECP ignores files with a TMP extension (both uppercase and lowercase) and files starting with a "." as the first character. Regexp pattern: *[A-Za-z0-9-]** Default file extension can be configured, which will be added to a received file in case that a file was sent without file extension.

Notes

- The content of each part of the filename can only contain alphanumeric characters and hyphens without accents, spaces and special characters, except for sender and receiver which can also contain the @ sign.
- ECP rejects a file
  - when the filename does not follow the format defined above;
  - when the filename length exceeds 255 characters;
  - when the file is empty.

To avoid read/write conflicts between ECP and BA, file should be copied or created with a ***.tmp*** extension or with a "." as the first character in a filename. When a file transfer is completed, rename it to the final name.

7.2. Folders

The Interface is formed by the following shared file system folders:

- OUT
  - contains: the message files to be sent via ECP; the file content can be arbitrary (XML, binary).
  - usage: a BA puts message files into the folder; ECP picks them up and removes them
- OUT_ERROR
  - contains: the message files that were not sent by ECP (either rejected or some error occurred during their processing).
  - usage: the files in this folder can be corrected manually and sent again.
- OUT_LOG
  - contains: the message log files.
  - The logged message events are accepted, refused, delivering, delivered, received, failed (the latter can only happen for a message that has been accepted by ECP).
  - usage: ECP continuously logs the message-related events in log files. When the message is accepted the message, ID is also logged; a BA can read the message log file at any time.
  - By configuration it is possible to create one file per message or one file per day.
- IN
  - contains: the received message files, the content of which can be arbitrary.
  - folder structure: subfolder for each message type or group of message types.
  - usage: ECP writes the received message files into the folder, BA picks up the message files (the BA is responsible for remove message files from the IN folder).
  - ECP writes received files with a ***.tmp*** extension, which is removed after a file writing is completed. Configure BA to ignore files with a ***.tmp*** extension to prevent read/write conflicts.

7.3. Message Send-Receive Scenario

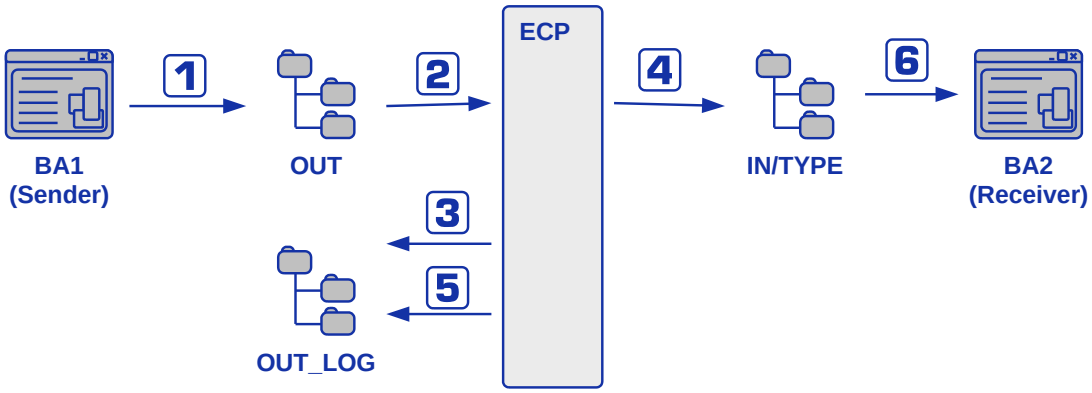

Figure 13: Message Send-Receive Scenario

Message send-receive scenario (with sender's endpoint configured to use one-per-message log):

1. The Sender BA puts the message file *"<SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.EXT"* into the OUT folder.
2. The ECP picks up the message file *"<SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.EXT"* from the OUT folder.
3. The ECP creates the log file *"<SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.log"* into the OUT_LOG folder and writes the "message accepted" record into it.
4. The ECP transports the message file through the ECP network and continuously logs message-related events in the *"<SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.log"* file.
5. On the recipient side, the ECP stores the message into the *"<SENDERAPP>_<SENDER>_<TYPE>_<BAMessageID>_<ECPMessageID>.EXT"* file into the IN folder and then writes "message received" record into the *"<SENDERAPP>_<RECEIVER>_<TYPE>_<BAMessageID>.log"* file on sender's side.
6. The Receiving BA picks up the message file *"<SENDERAPP>_<SENDER>_<TYPE>_<BAMessageID>_<ECPMessageID>.EXT"* from the IN folder.

8. Messaging API - AMQP

The AMQP API is high performant messaging API which can be used by the business application.

Note that the AMQP API channel only provides a subset of the messaging API operations and possibilities. The following operations are supported:

- Send message – without the support of the conversation ID
- Receive message – the "confirm message receive" operation is implicit. The message stated is changed to "received" immediately after the message is written in the Inbox queue.
- Check message status – status changes are pushed by ECP to dedicated queue.

Refer to the *ECP Administration Guide* for detailed information about AMQP API configuration.

8.1. Send Message

Send message is an operation that can be "called" by publishing AMQP message to *"ecp.endpoint.outbox"* queue. ECP takes the message, verifies it, and generate response/fault. The response/fault is published by ECP in *"ecp.endpoint.outbox.reply"* queue. When a message is received by the ECP, it must be possible to parse it as the *JmsBytesMessage.class* from the Apache QPID library. Otherwise, an exception will be thrown and the message will be discarded.

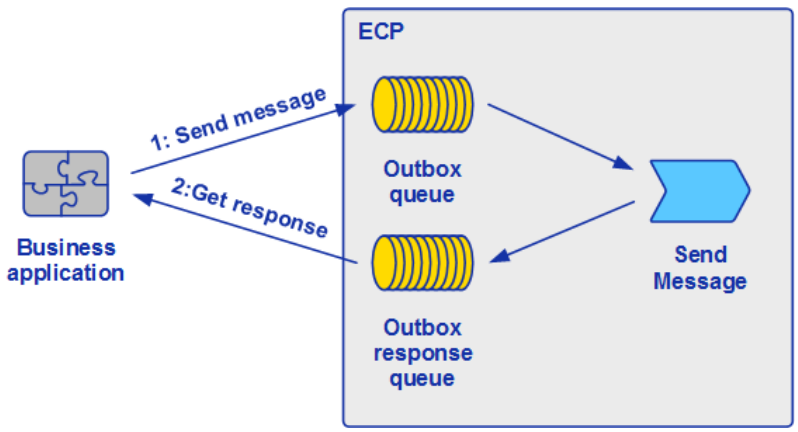

Figure 14: AMQP Send Message

8.1.1. Input Message Structure

8.1.1.1. Properties

Parameter	Type	Description	Required
JMSCorrelationID	string	Unique ID of request. ECP will generate response/fault only in case that JMSCorrelationID is set.	False

8.1.1.2. Application Properties

Parameter	Type	Description	Required
receiverCode	string	Endpoint code of the message's receiver.	True
messageType	string	Message business type.	True
baMessageID	string	Message ID assigned by BA.	False
senderApplication	string	Core or description of sender's BA.	False

8.1.1.3. Data

AMQP Message data section must contain the message payload.

8.1.2. Output Message Structure

8.1.2.1. Properties

Parameter	Type	Description	Required
JMSCorrelationID	string	Unique ID of request.	True

8.1.2.2. Application Properties

Parameter	Type	Description	Required
messageID	string	The ID of sent message. In case of error is this field empty.	True

8.1.3. Fault Message Structure

8.1.3.1. Properties

Parameter	Type	Description	Required
JMSCorrelationID	string	Unique ID of request.	True

8.1.3.2. Application Properties

Parameter	Type	Description	Required
Error code	string	The code representing the type of error (e.g. validation error, unexpected error, etc.).	True
Error ID	string	Unique identification of the failure. The Error ID shall always be used to track the error in the application logs.	True
Error message	string	An error message usually describing the reason of failure.	True
Error details	string	Additional error details.	False

8.2. Receive Message

Receiving of messages through AMQP is realized by consuming messages from one of Inbox queues. The "confirm message receive" operation is implicit. The message state is changed to "received" immediately after the message is written in the Inbox queue. ECP can be configured to receive only certain message types via AMQP and it can be configured to use different Inbox queue for each message type. Default inbox queue name is *"ecp.endpoint.inbox"*.

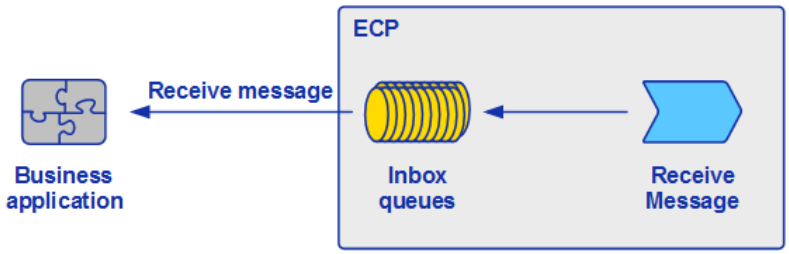

Figure 15: AMQP Receive Message

8.2.1. Message Structure

8.2.1.1. Application Properties

Parameter	Type	Description	Required
senderCode	string	Endpoint code of the message's sender.	True
businessType	string	Message business type.	True
baMessageID	string	Message ID assigned by BA.	False
senderApplication	string	Core or description of sender's BA.	False

8.2.1.2. Data

AMQP Message data section must contain the message payload.

8.3. Check Message Status

Check of message status through AMQP is realized by consuming statuses from a dedicated queue. The changes to status of a message sent through AMQP channel are pushed to the dedicated queue by a specialized sent handler of ECP (see chapter *Send Event Handler*). Application interested in the status of sent messages consumes the information from the queue. Default send event handler queue is *"ecp.endpoint.send.event".*

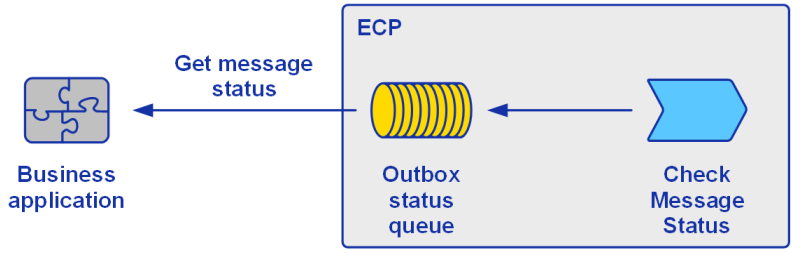

Figure 16: AMQP Check Message Status

8.3.1. Message Structure

8.3.1.1. Application Properties

Parameter	Type	Description	Required
messageID	string	Message ID of the sent message.	True

8.3.1.2. Data

AMQP Message data section contains details about the status change:

Parameter	Description
messageID	Message ID of the sent message.
senderCode	Endpoint code of the message's sender.
receiverCode	Endpoint code of the message's receiver.
businessType	Message business type.
senderApplication	Core or description of sender's BA.

baMessageID	Message ID assigned by BA.
messageState	Status of the message. It can be one of the following: VERIFYING ACCEPTED DELIVERING DELIVERED RECEIVED FAILED
sendTime	Date and time when the message was sent.
receiveTime	Date and time when the message was received.
trace	List of information events about the message's traversal through ECP network

Trace parameter contains an event list. Each event has the following parameters:

Parameter	Description
timestamp	Date and time of the event.
eventType	Message tracing status. It can be one of the following: VERIFYING ACCEPTED TRANSPORTED DELIVERED RECEIVED FAILED
component	The code of ECP messaging component on which the event happened.
componentDescription	A description of the messaging component.
eventDetails	Details about the event.

9. Management API Description

ECP provides a management API as a SOAP web service that can be used by business applications to adjust the configuration of Endpoint. This way, users of business applications (or processes within the business applications) can adjust the behaviour of ECP directly as required without having to access the ECP GUI.

Please refer to Attachment A – Web Service Public API WSDL for WSDL information.

9.1. Message Paths Management

Message path management web service provides a way to administer message paths of an Endpoint allowing its user to execute any operation that is available also via GUI.

9.1.1. Create Message Path

User can use the create message path operation to create a new message path to his endpoint. The created message path has receiver set automatically to the component code of the endpoint. Upon submission of the request, endpoint checks that the new path does not conflict with other rules as described in the ECP Functional Design.

The operation follows the semantics of “Create message path” use case on the GUI.

9.1.1.1. Input

CREATE MESSAGE PATH REQUEST:

Parameter	Type	Description	Required
messagePath	MessagePathChange	The message path that should be created.	True

MESSAGE PATH CHANGE:

Parameter	Type	Description	Required
messageType	string	Business type of messages that should be routed through this message path.	True

path	string	Defines the path of message in format {TYPE}:{COMPONENT_CODE} - TYPE is DIRECT or INDIRECT. - COMPONENT_CODE is used only for INDIRECT paths.	True
validFrom	dateTime	Date and time from which the message path can be used.	True
validTo	dateTime	Date and time to which the message path can be used. If not set, the message path is valid forever.	False
allowedSenders	List<string>	List of component codes of endpoints that can send messages using this message path. If not set, any endpoint is allowed as a sender.	False

9.1.1.2. Output

CREATE MESSAGE PATH RESPONSE:

Parameter	Type	Description	Required
messagePath	MessagePath	The message path that was created.	True

MESSAGE PATH:

Parameter	Type	Description	Required
id	string	Unique identifier of the message path.	True
status	string	Current status of the message path. Allowed values: ACTIVE, INVALID	True
messageType	string	Business type of messages that should be routed through this message path.	True

path	string	Defines the path of message in format {TYPE}:{COMPONENT_CODE} - TYPE is DIRECT or INDIRECT. - COMPONENT_CODE is used only for INDIRECT paths.	True
validFrom	dateTime	Date and time from which the message path can be used.	True
validTo	dateTime	Date and time to which the message path can be used. If not set, the message path is valid forever.	False
allowedSenders	List<string>	List of component codes of endpoints that can send messages using this message path.	False

9.1.2. Get Message Path List

The operation provides access to all message paths defined within the endpoint.

The operation follows the semantics of “Message paths list” use case on the GUI described in the Functional Specification.

9.1.2.1. Input

CREATE MESSAGE PATH LIST REQUEST:

Parameter	Type	Description	Required
status	string	Status of requested message paths functioning as a filter over the returned message path list. Allowed values: ACTIVE, INVALID	False

9.1.2.2. Output

CREATE MESSAGE PATH LIST RESPONSE:

Parameter	Type	Description	Required
messagePaths	List<MessagePath>	List of found message paths. For description of MessagePath type, see chapter *Message Paths Management* > *Output*.	True

9.1.3. Edit Message Path

Edit message path operation allows the user to adjust some attributes of an existing message path. The existing message path is identified by its id and new values are provided for all attributes. If the value for any attribute does not change, the attribute must be set to the original value. If an attribute is missing, the operation clears the value for given attribute on the existing message path.

The operation follows the semantics of “Edit message path” use case on the GUI described in the Functional Specification.

9.1.3.1. Input

EDIT MESSAGE PATH REQUEST:

Parameter	Type	Description	Required
messagePathId	string	Unique identifier of the edited message path.	True
messagePath	MessagePathChange	New values for the message path attribute. See chapter *Message Paths Management* > *Input* for the description of MessagePathChange type.	True

9.1.3.2. Output

EDIT MESSAGE PATH RESPONSE:

Parameter	Type	Description	Required
messagePath	MessagePath	The message path after applied changes. For description of MessagePath type, see chapter *Message Paths Management > Output*.	True

9.1.4. Delete Message Path

Delete operation removes an existing message path from the directory.

The operation follows the semantics of “Delete message path” use case on the GUI described in the Functional Specification.

9.1.4.1. Input

DELETE MESSAGE PATH REQUEST:

Parameter	Type	Description	Required
messagePathId	string	Unique identifier of the deleted message path.	True

9.1.4.2. Output

DELETE MESSAGE PATH RESPONSE:

Parameter	Type	Description	Required
messagePath	MessagePath	The message path that was deleted. For description of MessagePath type, see chapter *Message Paths Management > Output*.	True

9.2. Error Handling

Errors which occur on the server-side of web service (e.g. input data validation errors, unexpected errors, etc.) are reported back to the client using the standard SOAP Fault mechanism (http://www.w3.org/TR/soap12-part1/#soapfault). The ECP specific information about the error is transported using the Detail part of the SOAP Fault element.

Management API uses a generic management operation soap fault described in WSDL.

10. Plugins

This chapter describes interfaces related to ECP plugins.

Interface	Description
ReceiveEventHandler	The interface for handling ReceiveEvents. The interface is used to implement custom handlers for receive events. Each handler must be registered into the endpoint as described in the *ECP Administration Guide*. The implementation typically downloads the related message and pushes it directly into the business application or directly notifies the business application about the message waiting to be received.
ReceiveEvent	An event which indicates that a message or an acknowledgement was delivered to the endpoint.
SendEventHandler	The interface for handling SendEvents. The interface is used to implement custom handlers for send events. Each handler must be registered in the endpoint as described in the *ECP Administration Guide*.
SendEvent	An event which indicated that a message changed its state to DELIVERED, RECEIVED or FAILED.
ArchiveEventHandler	The interface for handling ArchiveEvents. The interface is used to implement custom handlers for archive events. Each handler must be registered in the endpoint as described in the *ECP Administration Guide*.
ArchiveEvent	An event which indicated that message content or metadata should be archived.

10.1. Receive Event Handler

This chapter contains a detailed description of ReceiveEventHandler and ReceiveEvent interfaces.

Domain model:

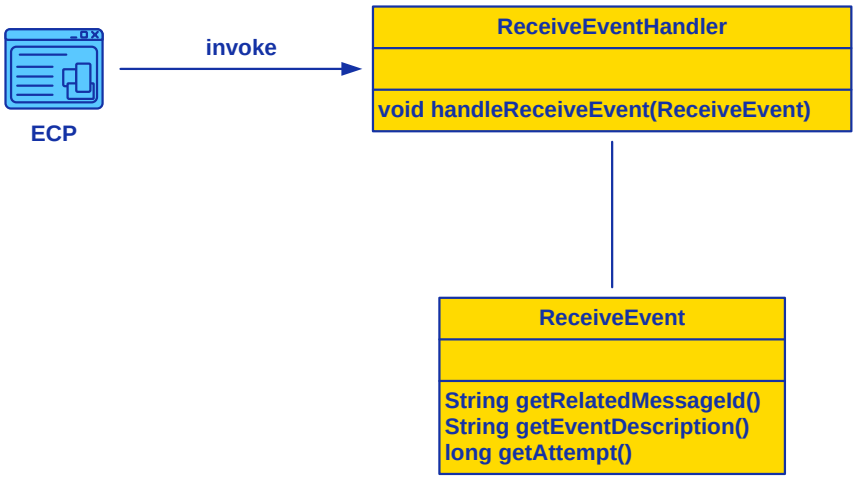

Figure 17: Receive Event Handler

RECEIVEEVENTHANDLER DESCRIPTION:

Method	Description
handleReceiveEvent	Invoked when a new message arrives at this endpoint and is waiting to be received. In a typical implementation of the handler, the event is processed in one of the following ways: - downloads the related message via ECPMessaging and pushes it directly to the business application; - or directly notifies the business application about the message waiting to be received. If this method throws an exception, ECP will try to call it again.

RECEIVEEVENT DESCRIPTION:

Method	Description
getRelatedMessageId	Returns ID of the message related to the ReceiveEvent.
getEventDescription	Returns text description of the ReceiveEvent.
getAttempt	Returns a sequence number of the attempt for notification with the ReceiveEvent. If processing of ReceiveEvent fails, the ECP tries to repeat the notification (with an incremented attempt field).

10.2. Send Event Handler

This chapter contains a detailed description of the SendEventHandler and SendEvent interfaces.

Domain model:

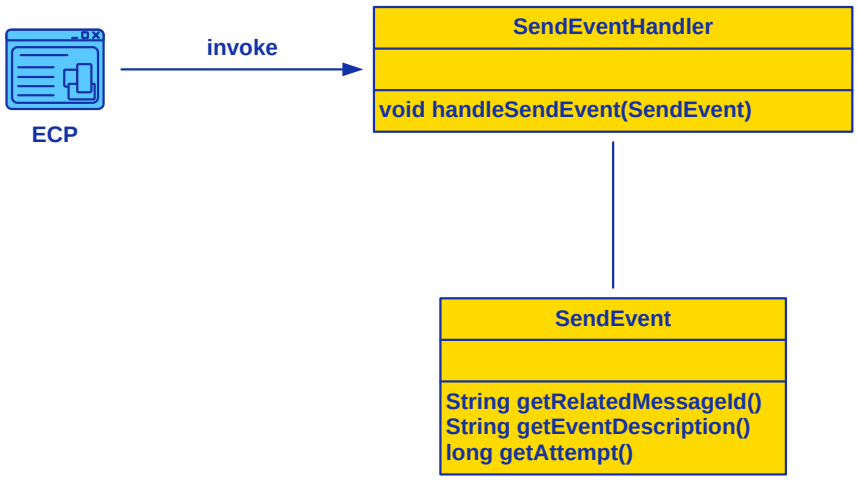

Figure 18: Send Event Handler

SENDEVENTHANDLER DESCRIPTION:

Method	Description
handleSendEvent	Invoked when the status of the sent message is changed to DELIVERED, RECEIVED of FAILED. If this method throws an exception, ECP will try to call it again.

SENDEVENT DESCRIPTION:

Method	Description
getRelatedMessageId	Returns ID of the message related to the SendEvent.
getEventDescription	Returns the status of the sent message. Possible values are: - delivered - received - failed

getAttempt	Returns sequence number of attempts for notification with the SendEvent. If the processing of SendEvent fails, the ECP reattempts the notification (with an incremented attempt field).

10.3. Archive Event Handler

This chapter contains a detailed description of the ArchiveEventHandler interface together with related structures.

Domain model:

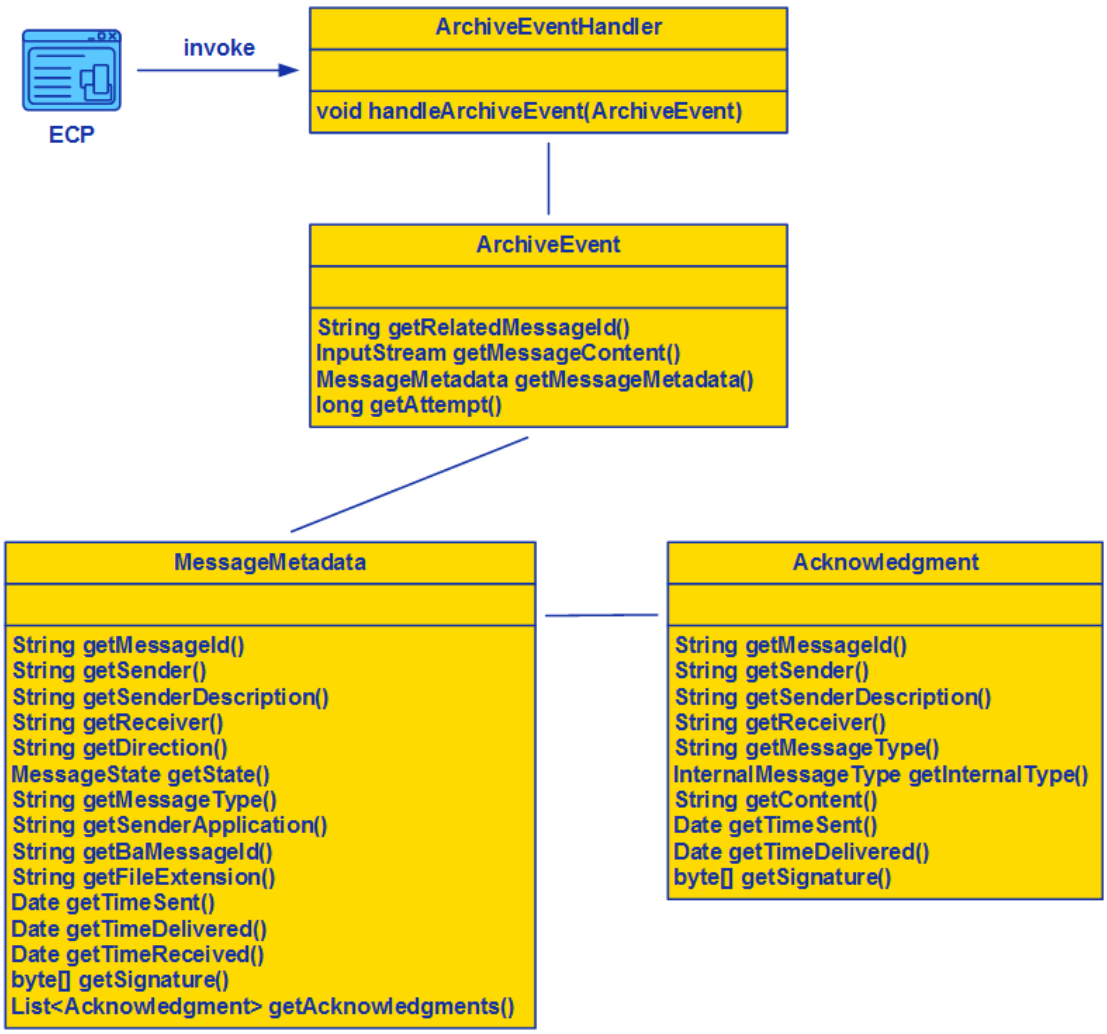

Figure 19: Archive Event Handler

ARCHIVEEVENTHANDLER DESCRIPTION:

Method	Description
handleArchiveEvent	Invoked whenever there is a message content or metadata ready to be moved to archive. If this method throws an exception, ECP will try to call it again.

ARCHIVEEVENT DESCRIPTION:

Method	Description	Optional
getRelatedMessageId	Returns ID of the message related to the ArchiveEvent.	false
getMessageConent	Binary content of the message.	true (*)
getMessageMetadata	Metadata of the message.	true (*)
getAttempt	Sequence number of attempts for notification with the ArchiveEvent. If the processing of ArchiveEvent fails, the ECP reattempts the notification (with an incremented attempt field).	false

(*) One of the fields is always filled

MESSAGEMETADATA DESCRIPTION:

Method	Description	Optional
messageId	ID of the message related to the ArchiveEvent	false
sender	Component code of the message sender	false
senderDescription	Human-readable description of the message sender	true
receiver	Component code of the message receiver	false
direction	Direction of the message (IN / OUT)	false
state	Final state of the message	false
messageType	Business type of the message	false
senderApplication	Business application that sent the message	true

baMessageId	Business application message ID	true
fileExtension	The extension of the sent file	true
timeSent	Time when the message was accepted at sender Endpoint	false
timeDelivered	Time when the message was delivered to receiver Endpoint	true
timeReceived	Time when the message was received by business application	true
signature	Digital signature of the message sender	true
acknowledgments	List of acknowledgments	false

ACKNOWLEDGMENT DESCRIPTION:

Method	Description	Optional
messageId	ID of the acknowledgment message	false
sender	Component code of the acknowledgment sender	false
senderDescription	Human-readable description of the acknowledgment sender	true
receiver	Component code of the acknowledgment receiver	false
messageType	Business type of the original message	false
internalType	Type of the acknowledgment (delivery, receive, failure)	false
content	Content of the acknowledgment	false
timeSent	Time when the acknowledgment was created	false
timeDelivered	Time when the acknowledgment was delivered	true
signature	Digital signature of the acknowledgment sender	true

10.4. Java API for Plugins

ECP provides a Java API exclusively to support the features of plugins. The ECP functions are accessed via classes and packages located in package *eu.entso_e.ecp.v1.endpoint*. Operations are invoked using standard java calls. The key components are:

Component	Description
ECPFacadeProvider	Contains a static method providing ECPMessaging.
ECPMessaging	The interface represents a facade providing access to the messaging functions of the ECP Module.

The operation of the messaging API can be invoked in following way:

1. Get ECPMessaging component using the ECPFacadeProvider component.
2. Invoke messaging operation.

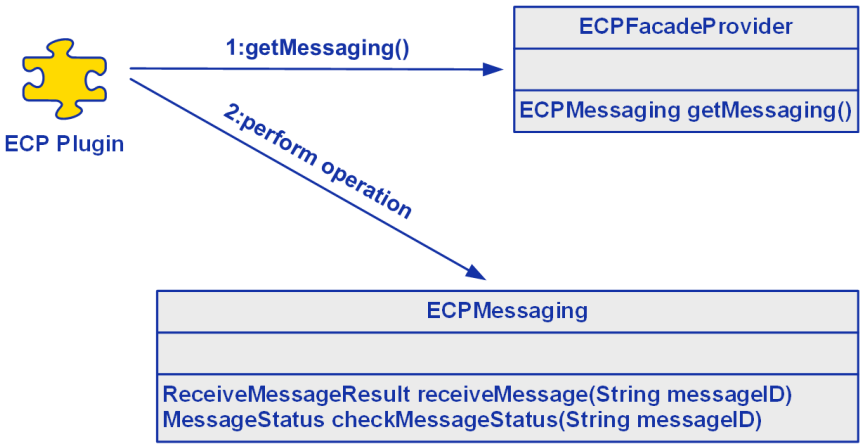

Figure 20: Java API for Plugins

10.4.1. Receive Message Method

This method provides the Receive Message operation described in chapter *Messaging API - Logical Description > Receive Message*.

Domain model of operation:

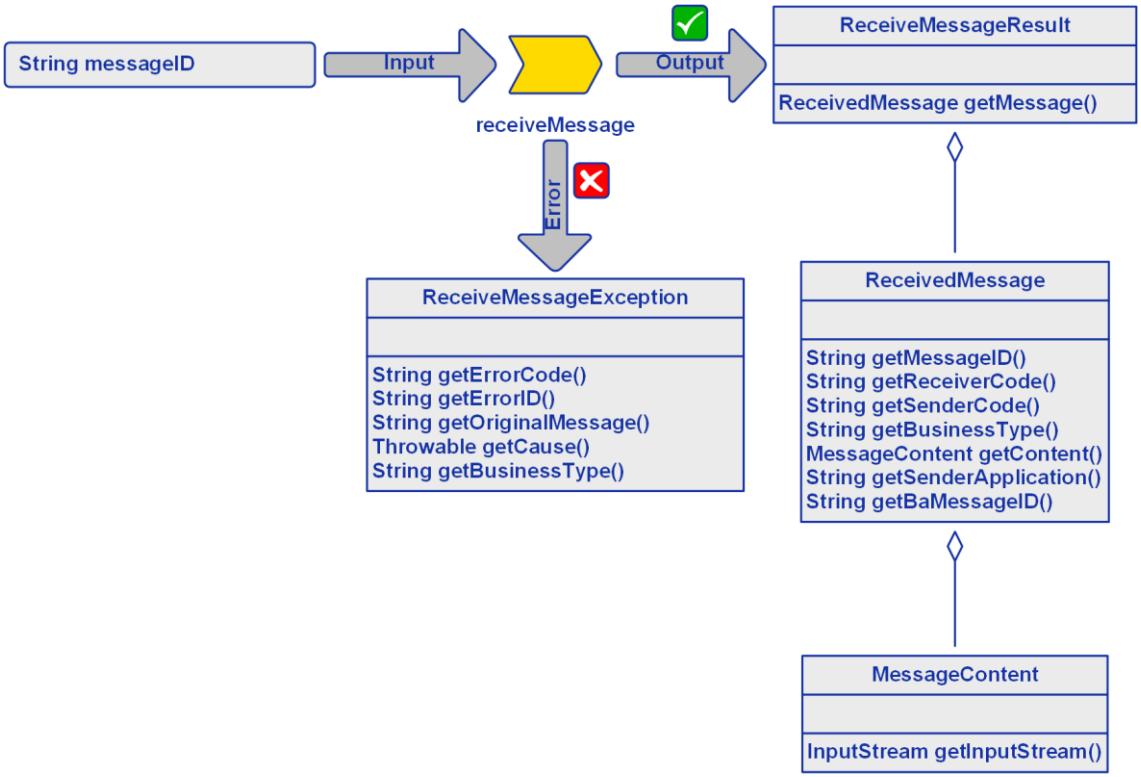

Figure 21: Plugins - Receive Message Method

10.4.2. Check Message Status Method

This method provides the Check Message Status operation described in chapter *Messaging API - Logical Description > Check Message Status*.

Domain model of operation:

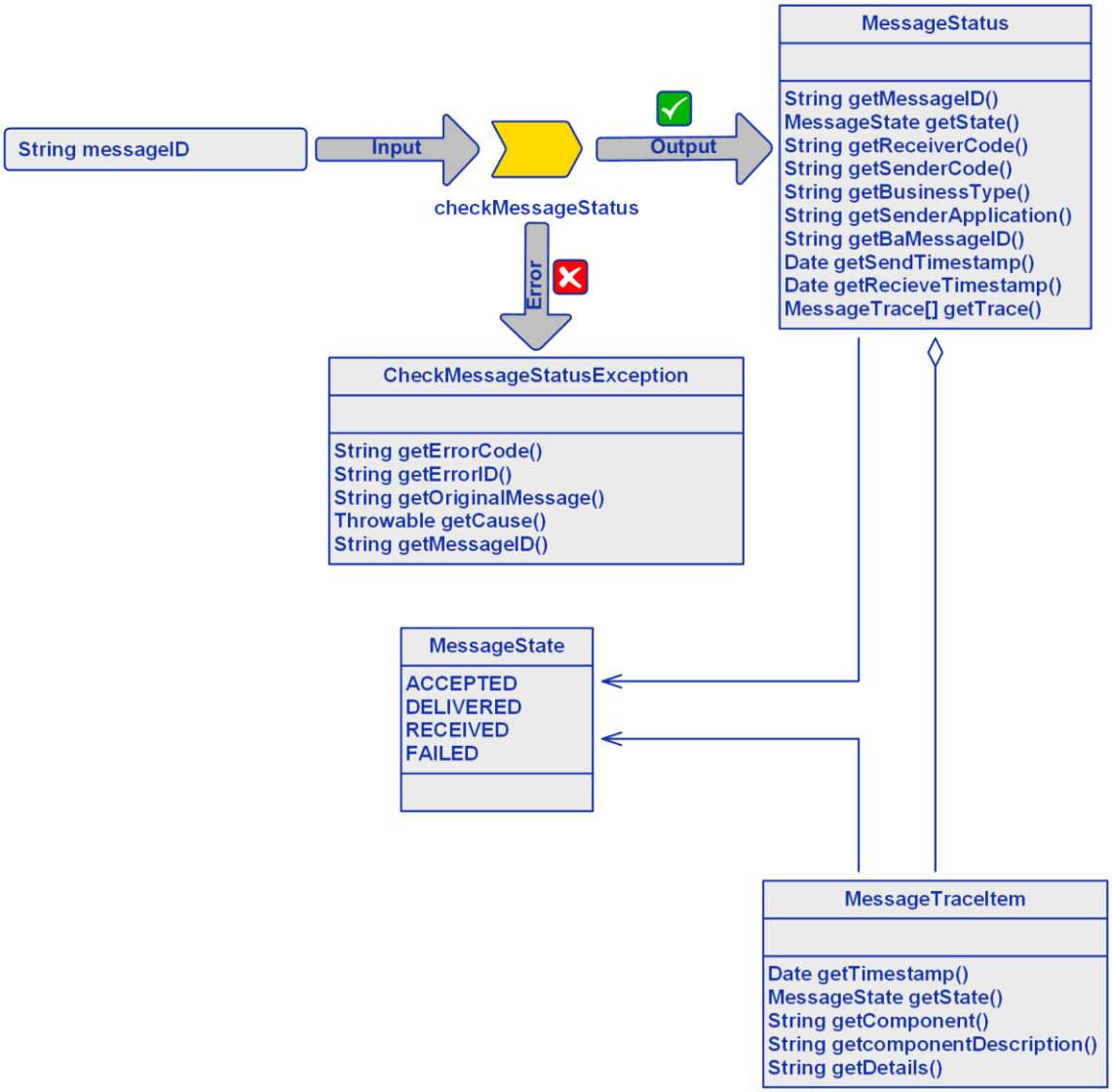

Figure 22: Plugins - Check Message Status Method

10.4.3. Error Handling

Every messaging operation provides its own exception and contains all the information described in chapter *Messaging API - Logical Description*. ECP reports the errors to plugins using standard Java exceptions mechanism. Exceptions thrown by the API provide methods described in the table below.

Method	Description
getErrorCode()	Provides error code.
getErrrorID()	Provides unique error ID.

getMessage()	Provides error message.
getCause()	Provides details of the error.

11. Configuration of Channels and Message Receivers

The Endpoint may use multiple channels and several message receivers for receiving messages. It is possible to configure them to accept only messages of specified types. Each message receiver can accept multiple message types, but each message type can be accepted by only one receiver.

For example, the FSSF channel can be configured to accept Type1 and Type2 messages, while some user's custom Receive Event Handler accepts Type3 messages.

The detailed configuration of E/D Client channels is described in the *ECP Administration Guide* document in chapter *ECP Public API – Configuration*.

12. Attachment A - Web Service Public API WSDL

WSDL documents can be downloaded from any running Endpoint using any web browser.

- Messaging web service
  - WSDL URL pattern:
    https://<endpointIP>:8443/ECP_MODULE/services/MadesEndpoint.wsdl
- Message Path Management web service
  - WSDL URL pattern:
    https://<endpointIP>:8443/ECP_MODULE/services/MessagePathsEndpointService.wsdl

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu

13. Figure Content

Figure 1: Receive Handlers 12
Figure 2: Messaging API - Logical Description 14
Figure 3: Send Message 15
Figure 4: Receive Message 17
Figure 5: Confirm Receive Message 19
Figure 6: Check Message Status 20
Figure 7: Connectivity Test 24
Figure 8: WebMethod Domain Model 27
Figure 9: WebMethod Domain Model 28
Figure 10: WebMethod Domain Model 29
Figure 11: WebMethod Domain Model 31
Figure 12: WebMethod Domain Model 32
Figure 13: Message Send-Receive Scenario 36
Figure 14: AMQP Send Message 37
Figure 15: AMQP Receive Message 40
Figure 16: AMQP Check Message Status 41
Figure 17: Receive Event Handler 50
Figure 18: Send Event Handler 51
Figure 19: Archive Event Handler 52
Figure 20: Java API for Plugins 55
Figure 21: Plugins - Receive Message Method 56
Figure 22: Plugins - Check Message Status Method 57

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www.entsoe.eu